

ClimaYa

Prototipo móvil de consulta del clima con la API pública de Open-Meteo

Institución	*[completar]*
Asignatura	*[completar]*
Actividad	Actividad 17 — Prototipo de aplicación móvil que consume una API pública
Alumno(a)	*[completar]*
Matrícula / Grupo	*[completar]*
Docente	*[completar]*
Fecha de entrega	*[completar]*
Nombre del prototipo	ClimaYa v1.0.0

Contenido

1. Análisis, selección de escenario y arquitectura
2. Identificación y documentación de la API
3. Medidas de seguridad implementadas
4. Casos de prueba
5. Capturas de pantalla para demostrar el funcionamiento
6. Documento de entrega
7. Anexo A. Instalación y ejecución

1. Análisis, selección de escenario y arquitectura

1.1 Requerimientos de la actividad

#	Requerimiento	Cómo se cubre en ClimaYa
R1	Prototipo móvil funcional	App Expo/React Native ejecutable en Android/iOS (Expo Go) y en navegador
R2	Consumir una API pública mediante HTTP GET	Dos endpoints GET de Open-Meteo (geocodificación y pronóstico)
R3	Procesar JSON y mostrar ≥ 3 campos por resultado	Búsqueda: nombre, estado/región, país, coordenadas, altitud, población. Clima: temperatura, sensación térmica, humedad, viento, precipitación, condición, pronóstico de 7 días
R4	Búsqueda y/o actualización de información	Búsqueda de ciudades (con búsqueda automática al escribir) + actualización del clima (botón y "deslizar para actualizar")
R5	Indicadores de carga, estados vacíos y errores	Componente `StateView` con variantes de carga, vacío, error (con "Reintentar") e informativo; aviso no bloqueante si falla una actualización
R6	HTTPS, sin credenciales expuestas	Solo URLs `https://` de una lista de hosts permitidos; la API no requiere clave
R7	Validación de parámetros	Validación de texto de búsqueda, coordenadas y parámetros de navegación
R8	Manejo de errores de conexión, respuestas inesperadas y límites	Errores tipados (red, timeout, 400, 429, 5xx, JSON inválido), reintento automático, limitador local y caché
R9	Condiciones de uso y atribución	Atribución CC BY 4.0 visible en ambas pantallas

1.2 Evaluación de escenarios

Las instrucciones que recibí no incluían la lista de escenarios propuestos de la actividad, así que comparé los escenarios más comunes para este tipo de práctica según los criterios que pide la actividad: API pública, gratuita, accesible por HTTPS y sin registro.

Escenario	API candidata	¿Requiere clave?	Gratuita	Búsqueda	Actualización	Veredicto
Clima / pronóstico	**Open-Meteo**	**No**	**Sí (uso no comercial)**	**Sí (geocodificación)**	**Sí (datos cada 15 min)**	**Seleccionado**
Clima	OpenWeatherMap	Sí	Plan limitado	Sí	Sí	Descartado: exige clave que quedaría expuesta en la app
Tipo de cambio	Frankfurter / ExchangeRate	No / Sí	Sí	Limitada	Diaria	Poca variedad de campos y de búsqueda
Películas	TMDB / OMDb	Sí	Sí	Sí	No	Exige clave
Noticias	NewsAPI	Sí	Solo desarrollo	Sí	Sí	Exige clave y no permite uso en cliente en producción

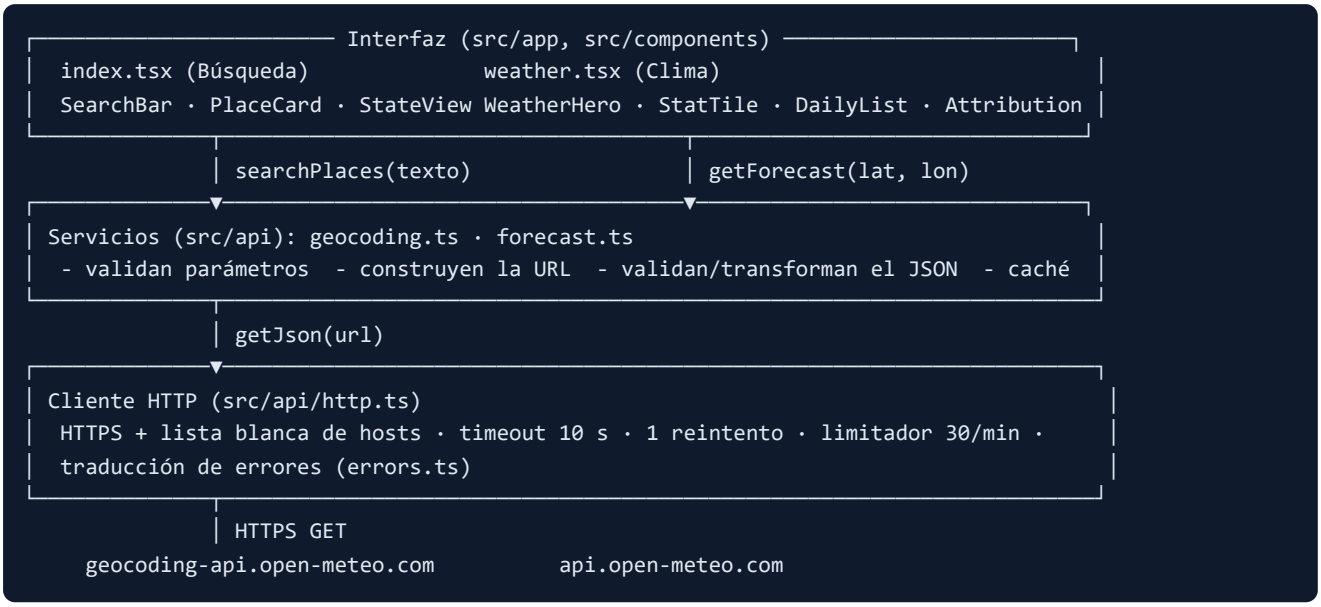
Justificación: Open-Meteo permite cubrir ambas funciones (búsqueda **y** actualización), devuelve JSON con muchos campos útiles, no necesita clave (así no hay credenciales que proteger dentro de la app) y su documentación es pública y detallada.

1.3 Stack tecnológico

Capa	Tecnología	Justificación
Framework móvil	**Expo SDK 58 + React Native 0.88**	Permite ejecutar el prototipo en un teléfono real con **Expo Go** (escaneando un código QR) sin instalar Android Studio ni Xcode; también corre en navegador para demostraciones
Lenguaje	**TypeScript (modo estricto)**	Tipos para las respuestas de la API y menos errores en tiempo de ejecución
Navegación	**Expo Router** (rutas por archivos en `src/app/`)	Solución recomendada por Expo; pila nativa con botón "atrás"
HTTP	**`fetch` nativo + `AbortController`**	Sin dependencias extra; permite timeouts y cancelar solicitudes obsoletas
Pruebas	**Jest + jest-expo**	Pruebas unitarias (lógica, validación, manejo de errores) y de integración contra la API real

No se añadieron librerías de terceros con código nativo, así que la app funciona en Expo Go sin compilar.

1.4 Arquitectura



Estructura del proyecto

```
├── app.json                Configuración Expo (nombre, esquema, permisos)
├── package.json            Dependencias y scripts
├── jest.config.js          Configuración de pruebas
├── src/
│   ├── app/                Pantallas (Expo Router)
│   │   ├── _layout.tsx     Pila de navegación y estilo del encabezado
│   │   ├── index.tsx       Pantalla de búsqueda de ciudades
│   │   ├── weather.tsx     Pantalla de clima actual + pronóstico
│   │   └── +not-found.tsx  Ruta desconocida
│   ├── api/
│   │   ├── http.ts         Cliente GET seguro (HTTPS, timeout, reintentos)
│   │   ├── errors.ts       Errores tipados y mensajes al usuario
│   │   ├── rateLimiter.ts  Límite local de solicitudes
│   │   ├── cache.ts        Caché en memoria con expiración
│   │   ├── geocoding.ts    Búsqueda de ciudades
│   │   └── forecast.ts     Clima actual y pronóstico
│   ├── components/         Componentes visuales reutilizables
│   ├── utils/
│   │   ├── validation.ts   Validación de entradas
│   │   └── weather.ts      Códigos WMO y formato en español
│   └── theme.ts            Colores, espaciados y sombras
├── __tests__/              Pruebas unitarias
├── integration/            Pruebas de integración contra la API real
└── docs/                   Documentación de entrega
```

Flujo de la aplicación

1. El usuario escribe una ciudad. Tras 600 ms sin teclear (o al pulsar **Buscar**) se valida el texto y se consulta la API de geocodificación.
2. Se muestra la lista de coincidencias con nombre, región, país, coordenadas, altitud y población.
3. Al tocar un resultado se abre la pantalla de clima con las coordenadas. Estos parámetros se vuelven a validar.
4. Se consulta la API de pronóstico y se muestra el clima actual, cuatro indicadores y el pronóstico de 7 días.
5. El usuario puede **actualizar** con el botón o deslizando hacia abajo. Si la actualización falla, se conservan los últimos datos y aparece un aviso.

2. Identificación y documentación de la API

2.1 Ficha de la API

Concepto	Detalle
Nombre	**Open-Meteo** (Weather Forecast API + Geocoding API)
Proveedor	Open-Meteo.com (proyecto de código abierto, Patrick Zippenfenig)
Tipo	API REST pública, respuestas JSON
Protocolo	HTTPS (TLS)
Método usado	`GET`
Autenticación	**Ninguna** para uso no comercial (no se requiere clave)
Documentación oficial	Pronóstico: https://open-meteo.com/en/docs · Geocodificación: https://open-meteo.com/en/docs/geocoding-api
Términos de uso	https://open-meteo.com/en/terms
Licencia de los datos	**CC BY 4.0** — https://open-meteo.com/en/licence
Fuentes de datos	Modelos de servicios meteorológicos nacionales (DWD, NOAA, ECMWF, Météo-France, etc.); ciudades de la base **GeoNames**
Costo	Gratuita para uso no comercial

2.2 Condiciones de uso y límites (plan gratuito)

Límite	Valor
Por minuto	600 llamadas
Por hora	5 000 llamadas
Por día	10 000 llamadas
Por mes	300 000 llamadas
Uso permitido	Solo **no comercial** (proyectos personales, educativos, de investigación)
Al exceder límites	Los límites se aplican por IP. Open-Meteo puede bloquear aplicaciones o IP que abusen del servicio

Atribución obligatoria (CC BY 4.0): se debe dar crédito a Open-Meteo, enlazar la licencia y colocar un enlace junto a los datos mostrados (texto sugerido: "*Weather data by Open-Meteo.com*"). ➡️ ClimaYa muestra en ambas pantallas: "*Datos meteorológicos por **Open-Meteo.com** · **CC BY 4.0***" (pantalla de clima) y "*Búsqueda de ciudades por **Open-Meteo.com** (base GeoNames) · **CC BY 4.0***" (pantalla de búsqueda), con enlaces a ambos sitios.

Uso en este proyecto: es académico y no comercial, así que cumple las condiciones del plan gratuito.

2.3 Endpoint 1 — Búsqueda de ciudades (Geocoding)

```
GET https://geocoding-api.open-meteo.com/v1/search
```

Parámetro	Tipo	Obligatorio	Valor usado	Descripción
`name`	string	Sí	Texto validado del usuario	Nombre de la ciudad o código postal. Con 2 caracteres busca coincidencia exacta y con 3 o más busca por prefijo
`count`	entero	No	`10`	Número de resultados (1–100)
`language`	string	No	`es`	Idioma de los nombres de país y región
`format`	string	No	`json`	Formato de respuesta

Ejemplo de solicitud:

```
https://geocoding-api.open-meteo.com/v1/search?name=Toluca&count=10&language=es&format=json
```

Ejemplo de respuesta (resumida, obtenida en vivo):

```
{
  "results": [
    {
      "id": 3515302,
      "name": "Toluca de Lerdo",
      "latitude": 19.28786,
      "longitude": -99.65324,
      "elevation": 2671.0,
      "country_code": "MX",
      "timezone": "America/Mexico_City",
      "population": 489333,
      "country": "México",
      "admin1": "Estado de México"
    }
  ],
  "generationtime_ms": 0.288
}
```

> ⚠️ **Particularidad:** si no hay coincidencias, la API **omite** la propiedad `results` (responde `{"generationtime_ms": 0.04}`) en lugar de enviar un arreglo vacío. La app lo trata como "sin resultados".

Campos que muestra la app por resultado: `name`, `admin1` (estado), `country`, `country_code` (bandera), `latitude` / `longitude`, `elevation` y `population`.

2.4 Endpoint 2 — Clima actual y pronóstico (Forecast)

```
GET https://api.open-meteo.com/v1/forecast
```

Parámetro	Valor usado	Descripción
`latitude`	p. ej. `19.2879`	Latitud (−90 a 90)
`longitude`	p. ej. `-99.6532`	Longitud (−180 a 180)

Parámetro	Valor usado	Descripción
`current`	`temperature_2m,relative_humidity_2m,apparent_temperature,is_day,precipitation,weather_code,wind_speed_10m`	Variables del clima actual
`daily`	`weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max`	Variables diarias
`timezone`	`auto`	Horas en la zona horaria local de la ubicación
`forecast_days`	`7`	Días de pronóstico

Ejemplo de respuesta (resumida, obtenida en vivo):


```
{
  "timezone": "America/Mexico_City",
  "current": {
    "time": "2026-10-31T09:30",
    "temperature_2m": 16.9,
    "relative_humidity_2m": 42,
    "apparent_temperature": 14.9,
    "is_day": 1,
    "precipitation": 0.0,
    "weather_code": 3,
    "wind_speed_10m": 6.1
  },
  "daily": {
    "time": ["2026-10-31", "2026-11-01", "2026-11-02"],
    "weather_code": [3, 2, 1],
    "temperature_2m_max": [23.3, 22.8, 23.1],
    "temperature_2m_min": [9.9, 9.1, 8.9],
    "precipitation_probability_max": [0, 2, 0]
  }
}
```


Campos que muestra la app:

Campo JSON	Se muestra como	Unidad
`current.temperature_2m`	Temperatura actual (grande)	°C
`current.weather_code`	Condición en español + ícono (tabla WMO)	—
`current.apparent_temperature`	Sensación térmica	°C
`current.relative_humidity_2m`	Humedad	%
`current.wind_speed_10m`	Viento	km/h
`current.precipitation`	Precipitación	mm
`current.is_day`	Tarjeta en color de día/noche e ícono 🌙	—
`current.time`	Hora local de la medición	—
`daily.*`	Pronóstico de 7 días: día, ícono, probabilidad de lluvia, mínima y máxima	°C / %

Los datos de "current" se actualizan cada **15 minutos** (`"interval": 900`).

2.5 Códigos de respuesta y errores

Código	Situación	Cuerpo	Tratamiento en la app
200	Éxito	JSON con datos	Se valida la estructura y se muestra
200 sin `results`	Búsqueda sin coincidencias	`{"generationtime_ms": …}`	Estado vacío "Sin resultados para …"
400	Parámetro inválido	`{"error": true, "reason": "Latitude must be in range of -90 to 90°. Given: 999.0."}` (verificado en vivo)	Error `bad_request`: "El servicio rechazó la consulta…"
429	Límite de solicitudes excedido	—	Error `rate_limit`: "Espera N segundos…" (usa `Retry-After` si viene)
5xx	Falla del servidor	—	1 reintento automático; si persiste: "El servicio no está disponible…"
—	Sin conexión / DNS / TLS	—	Error `network`: "No hay conexión…" + botón Reintentar
—	Más de 10 s sin respuesta	—	Error `timeout` + reintento automático
—	JSON mal formado o campos faltantes	—	Error `invalid_response`: "Se recibió una respuesta inesperada…"

2.6 Tabla de códigos WMO (`weather_code`)

Código	Condición	Código	Condición
0	Despejado	61/63/65	Lluvia ligera/moderada/fuerte
1	Mayormente despejado	66/67	Lluvia helada
2	Parcialmente nublado	71/73/75	Nevada ligera/moderada/fuerte
3	Nublado	77	Granos de nieve
45/48	Niebla	80/81/82	Chubascos
51/53/55	Llovizna	85/86	Chubascos de nieve
56/57	Llovizna helada	95/96/99	Tormenta (con o sin granizo)

3. Medidas de seguridad implementadas

#	Riesgo	Medida	Dónde
S1	Datos interceptados o alterados en tránsito	**Solo HTTPS.** `assertAllowedUrl()` rechaza cualquier URL que no empiece con `https://` *antes* de llamar a `fetch`. La app no habilita tráfico HTTP sin cifrar	`src/api/http.ts`
S2	Conexiones a servidores no autorizados	**Lista blanca de hosts:** solo `api.open-meteo.com` y `geocoding-api.open-meteo.com`. Se comprueba el host exacto, así que `api.open-meteo.com.evil.com` también se rechaza	`src/api/http.ts` (`ALLOWED_HOSTS`)
S3	Exposición de claves o credenciales	La API elegida **no requiere clave**. No hay secretos, tokens ni archivos `.env` en el código ni en el paquete de la app. `.gitignore` excluye `.env*.local`, `*.key`, `*.p12`, `*.jks` y `*.pem`	Diseño, `.gitignore`
S4	Entradas maliciosas o malformadas	**Validación de búsqueda:** se recortan espacios; se exigen 2–60 caracteres; se usa una lista blanca de caracteres (letras con acentos/ñ, dígitos, espacios y `. , ' -`); se rechazan `<`, `>`, `;`, `&`, `=`, `%`, `/`, emojis y textos sin letras	`src/utils/validation.ts`
S5	Manipulación de parámetros de la URL	Los parámetros se codifican con `encodeURIComponent`, así que una entrada como `a&count=100` no puede añadir parámetros extra	`buildUrl()` en `src/api/http.ts`
S6	Enlaces profundos manipulados (`climaya://weather?lat=…`)	La pantalla de clima **vuelve a validar** latitud (−90..90) y longitud (−180..180) con una expresión regular numérica estricta, limpia caracteres de control y recorta los textos. Si algo es inválido, muestra "Ubicación inválida" sin llamar a la API	`validateLocationParams()`
S7	Respuestas inesperadas o maliciosas	**Validación del esquema JSON:** se comprueba el tipo de cada campo usado; los registros incompletos se descartan; las respuestas mal formadas producen un error controlado en lugar de un fallo de la app. Los datos se muestran como texto plano (React Native no interpreta HTML)	`parseGeocodingResponse()`, `parseForecastResponse()`
S8	Saturar el servicio o exceder los límites de la API	**Limitador local** de 30 solicitudes/min (muy por debajo de las 600/min permitidas), **caché** (10 min para búsquedas, 5 min para clima), **retardo de 600 ms** al escribir y cancelación de búsquedas obsoletas con `AbortController`	`rateLimiter.ts`, `cache.ts`, `index.tsx`
S9	Respuesta HTTP 429 (límite excedido)	Se detecta y se informa al usuario cuántos segundos esperar (lee `Retry-After` si existe). No se reintenta automáticamente para no empeorar el bloqueo	`http.ts`
S10	Solicitudes que nunca terminan	Timeout de 10 s con `AbortController`; un solo reintento con espera progresiva (solo ante fallas de red, timeout o 5xx)	`http.ts`
S11	Filtración de información técnica	Los errores internos se convierten en mensajes genéricos en español; nunca se muestran trazas, URLs ni respuestas crudas	`toUserMessage()` en `errors.ts`
S12	Permisos innecesarios	Solo se usa `INTERNET`. En `app.json` se bloquean explícitamente ubicación, cámara y micrófono	`app.json`
S13	Datos personales	La app no recopila, guarda ni envía datos personales: solo el nombre de ciudad que se busca y sus coordenadas públicas	Diseño

#	Riesgo	Medida	Dónde
S14	Cumplimiento de la licencia	Atribución CC BY 4.0 con enlaces a Open-Meteo y a la licencia en ambas pantallas	`src/components/Attribution.tsx`

Verificación

Todas las medidas S1, S2, S4–S11 tienen pruebas automatizadas en `__tests__/` (unitarias) e `integration/` (contra la API real). Ver la sección de ejecución en el `README.md`.

4. Casos de prueba

Datos de la ejecución

Concepto	Detalle
Aplicación	ClimaYa v1.0.0 (Expo SDK 58 / React Native 0.88)
Fecha de ejecución	31/10/2026, de 10:18:40 a 10:19:07 (hora del centro de México)
Entorno	Versión web de la app servida por Expo (`npm run web`), en Microsoft Edge 156 con vista móvil táctil de 390 × 844 px
API	Open-Meteo en producción (sin simulaciones; respuestas reales)
Evidencias	Capturas en `evidencias/casos-de-prueba/` y registro de solicitudes HTTP en `evidencias/casos-de-prueba/registro-ejecucion.json`

> **Nota sobre el entorno:** ambos casos se ejecutaron con la versión web de la app en vista móvil, porque el equipo de desarrollo no tiene emulador Android. El código de pantallas, validación y conexión es el mismo que se ejecuta en el teléfono. La pérdida de conexión del CP-02 se simuló con el modo sin conexión del navegador, que equivale al modo avión del teléfono. Se recomienda repetir ambos casos en un teléfono con Expo Go antes de la entrega (ver la sección final).

CP-01 — Búsqueda de una ciudad y consulta y actualización de su clima

Elemento	Descripción
Identificador	CP-01
Nombre	Búsqueda de una ciudad y consulta y actualización de su clima
Tipo	Prueba funcional de integración con la API (camino feliz)
Objetivo	Comprobar que la app consulta la API de Open-Meteo mediante solicitudes HTTP GET por HTTPS, procesa el JSON y muestra correctamente los datos de búsqueda y de clima, y que el botón "Actualizar" obtiene información nueva.
Justificación	Es el flujo principal del prototipo y cubre los requerimientos centrales de la actividad: integración con una API pública por GET y HTTPS, procesamiento de JSON con al menos tres campos por resultado, búsqueda, actualización de información, indicador de carga y atribución de la fuente. Si este caso falla, la app no cumple su propósito.

Precondiciones

1. Dependencias instaladas (`npm install`) y app en ejecución (`npm start` en el teléfono o `npm run web` en el navegador).
2. El dispositivo tiene conexión a internet.
3. La API de Open-Meteo está disponible (`https://geocoding-api.open-meteo.com` y `https://api.open-meteo.com`).
4. La app está recién abierta, en la pantalla de búsqueda, sin consultas previas.

Datos de prueba

Dato	Valor
Texto de búsqueda	`Toluca`
Resultado a seleccionar	Toluca de Lerdo, Estado de México, México

Pasos de ejecución

#	Acción
1	Abrir la aplicación
2	Escribir `Toluca` en el buscador y pulsar **Buscar**
3	Revisar los datos de la tarjeta "Toluca de Lerdo"
4	Tocar la tarjeta "Toluca de Lerdo"
5	Revisar los datos de clima mostrados y desplazarse hasta el pronóstico de 7 días
6	Pulsar ↻ **Actualizar**

Resultado esperado

#	Resultado esperado
1	Se muestra la pantalla "ClimaYa" con el buscador, un mensaje inicial y ciudades sugeridas
2	Aparece el indicador de carga "Buscando ciudades…". Se envía un `GET` por HTTPS a la API de geocodificación y se muestra una lista de resultados con su número total
3	La tarjeta muestra al menos tres campos y coinciden con el JSON: nombre, estado, país, coordenadas, altitud y población
4	Se envía un `GET` por HTTPS a la API de pronóstico (HTTP 200) y se abre la pantalla de clima de Toluca de Lerdo
5	Se muestran temperatura, condición, sensación térmica, humedad, viento y precipitación, con valores iguales (redondeados) a los del JSON. También aparecen el pronóstico de 7 días y la atribución "Datos meteorológicos por Open-Meteo.com · CC BY 4.0"
6	Se envía un nuevo `GET` (HTTP 200) y cambia la hora de "Actualizado en la app"

Resultado obtenido

#	Resultado obtenido	Evidencia
1	Se mostró la pantalla de búsqueda con el mensaje "Consulta el clima de cualquier ciudad" y 4 sugerencias	`CP01-01-pantalla-inicial.png`
2	Se mostró el indicador "Buscando ciudades… Consultando "Toluca"" y el botón Buscar se deshabilitó durante la carga. Solicitud: `GET https://geocoding-api.open-meteo.com/v1/search?name=Toluca&count=10&language=es&format=json` → **HTTP 200**. Se mostró "10 resultados para "Toluca"" (la API devolvió 10 registros)	`CP01-02-cargando.png`, `CP01-03-resultados.png`
3	Pantalla: *MX · Toluca de Lerdo · Estado de México, México · 19.29° N, 99.65° O · 2671 m · 489,333 hab.* JSON: `name=Toluca de Lerdo`, `admin1=Estado de México`, `country=México`, `latitude=19.28786`, `longitude=-99.65324`, `elevation=2671`, `population=489333`. **Coinciden los 7 campos**	`CP01-03-resultados.png`

#	Resultado obtenido	Evidencia
4	Solicitud: `GET https://api.open-meteo.com/v1/forecast?latitude=19.2879&longitude=-99.6532&current=…&daily=…&timezone=auto&forecast_days=7` → **HTTP 200**. Se abrió la pantalla "Toluca de Lerdo"	`CP01-04-clima.png`
5	Comparación pantalla / JSON: temperatura **19 °C** / `18.7`; condición **Parcialmente nublado** / `weather_code=2`; humedad **38 %** / `38`; viento **6 km/h** / `6`; precipitación **0.0 mm** / `0`; sensación térmica **18°** / `17.5`. Pronóstico del 31/10/2026 al 06/11/2026 (7 días). Atribución visible	`CP01-04-clima.png`, `CP01-05-pronostico-7-dias.png`
6	Se envió un nuevo `GET` a la API de pronóstico → **HTTP 200**. "Actualizado en la app" cambió de **10:19:00** a **10:19:03**. No hubo errores de JavaScript	`CP01-06-actualizado.png`

Estado: ✅ APROBADA

Todos los resultados obtenidos coincidieron con los esperados (16/16 verificaciones).

CP-02 — Manejo de pérdida de conexión y recuperación

Elemento	Descripción
Identificador	CP-02
Nombre	Manejo de pérdida de conexión y recuperación
Tipo	Prueba de manejo de errores (camino alterno / negativo)
Objetivo	Comprobar que, sin conexión a internet, la app no se cierra ni muestra errores técnicos: informa al usuario con un mensaje claro, conserva los datos ya obtenidos, ofrece reintentar y se recupera al volver la conexión.
Justificación	En un teléfono la pérdida de conexión es frecuente (túneles, zonas sin cobertura, cambio de Wi-Fi a datos). La actividad exige manejar errores de conexión e incluir mensajes de error. Este caso valida esa robustez y la medida de seguridad S11: no mostrar información técnica al usuario.

Precondiciones

1. Haber ejecutado CP-01: la app muestra el clima de **Toluca de Lerdo** con datos cargados.
2. El dispositivo tiene conexión a internet al iniciar la prueba.
3. Se puede desactivar la conexión: modo avión en el teléfono o modo sin conexión en el navegador.

Datos de prueba

Dato	Valor
Ciudad con datos cargados	Toluca de Lerdo
Texto de búsqueda sin conexión	`Monterrey`

Pasos de ejecución

#	Acción
1	Verificar que la pantalla de clima de Toluca de Lerdo muestra datos

#	Acción
2	Desactivar la conexión a internet (modo avión)
3	Pulsar ↻ **Actualizar**
4	Volver a la búsqueda con ←, escribir `Monterrey` y pulsar **Buscar**
5	Reactivar la conexión y pulsar **Reintentar**

Resultado esperado

#	Resultado esperado
1	Se muestran los datos del clima
2	La app sigue abierta, sin cambios visibles
3	Aparece el aviso "No hay conexión con el servicio… Se muestran los últimos datos obtenidos". Los datos previos se conservan y la app no se cierra
4	Aparece el estado de error "No se pudo completar la búsqueda" con un mensaje comprensible (sin trazas técnicas) y el botón **Reintentar**
5	La búsqueda se completa (HTTP 200) y se muestran los resultados de Monterrey

Resultado obtenido

#	Resultado obtenido	Evidencia
1	Se mostró la temperatura **19 °C** y los demás datos del clima	`CP01-06-actualizado.png`
2	Se activó el modo sin conexión. La app permaneció abierta	—
3	Apareció el aviso: "⚠️ *No hay conexión con el servicio. Revisa tu conexión a internet e intenta de nuevo. Se muestran los últimos datos obtenidos.*" La temperatura siguió en **19 °C** (antes y después). El registro HTTP muestra **2 intentos fallidos** (`net::ERR_INTERNET_DISCONNECTED`): el intento original y el reintento automático	`CP02-03-aviso-sin-conexion.png`
4	Se mostró "*No se pudo completar la búsqueda — No hay conexión con el servicio. Revisa tu conexión a internet e intenta de nuevo.*" con el botón **Reintentar**. Hubo 2 intentos fallidos registrados, sin mensajes técnicos en pantalla	`CP02-04-error-busqueda.png`
5	Al pulsar Reintentar: `GET …/v1/search?name=Monterrey…` → **HTTP 200**. Primer resultado: *Monterrey · Estado de Nuevo León, México · 25.68° N, 100.32° O · 560 m · 1,135,512 hab.* No hubo errores de JavaScript durante la prueba	`CP02-05-recuperado.png`

Estado: ✅ APROBADA

Todos los resultados obtenidos coincidieron con los esperados (6/6 verificaciones).

Resumen

ID	Nombre	Tipo	Verificaciones	Estado
CP-01	Búsqueda de una ciudad y consulta y actualización de su clima	Funcional / integración con API	16/16	✅ Aprobada

ID	Nombre	Tipo	Verificaciones	Estado
CP-02	Manejo de pérdida de conexión y recuperación	Manejo de errores	6/6	✅ Aprobada

Pruebas automatizadas complementarias

Además de los dos casos documentados, el proyecto incluye pruebas automatizadas que se pueden volver a ejecutar en cualquier momento:

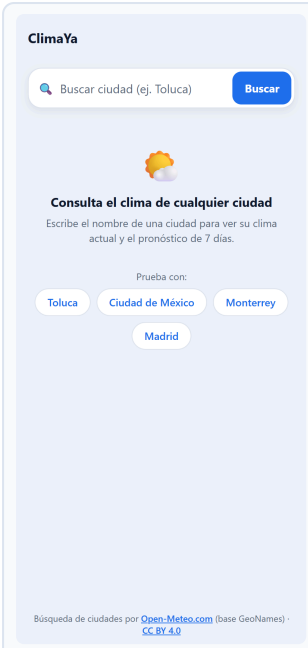
Conjunto	Comando	Qué cubre	Resultado (31/10/2026)
Unitarias	`npm test`	Validación de entradas (texto, coordenadas, caracteres maliciosos); solo HTTPS y lista blanca de hosts; codificación de parámetros; errores 400, 429, 5xx, red, timeout y JSON inválido; reintentos; limitador de solicitudes; lectura del JSON	62/62 aprobadas
Integración	`npm run test:integration`	Llamadas reales a ambos endpoints de Open-Meteo, búsqueda sin coincidencias, HTTP 400 real ante coordenadas imposibles	7/7 aprobadas

Repetición en dispositivo físico (recomendada)

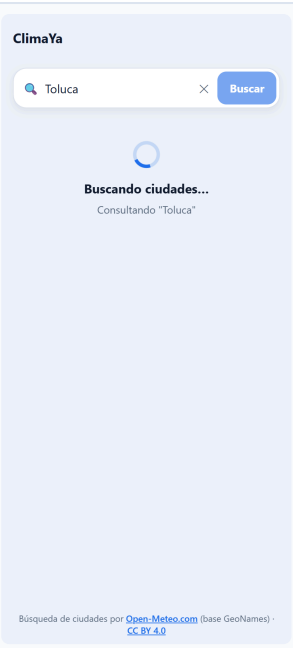
Para entregar evidencias desde un teléfono real, sigue los mismos pasos con Expo Go (`npm start` y escanear el código QR). Para el paso 2 del CP-02, activa el **modo avión**. Toma una captura en cada paso y anota cualquier diferencia en la columna "Resultado obtenido".

Caso	Fecha	Dispositivo / SO	Resultado	Estado
CP-01				
CP-02				

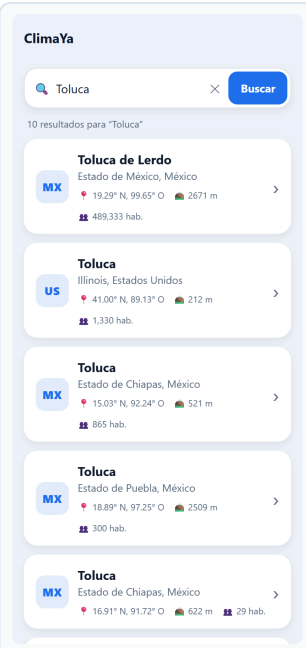
Evidencias de la ejecución



CP01-01-pantalla-inicial
CP-01 paso 1: pantalla inicial



CP01-02-cargando
CP-01 paso 2: indicador de carga



CP01-03-resultados
CP-01 pasos 2–3: resultados de "Toluca"



CP01-04-clima
CP-01 pasos 4–5: clima de Toluca de Lerdo



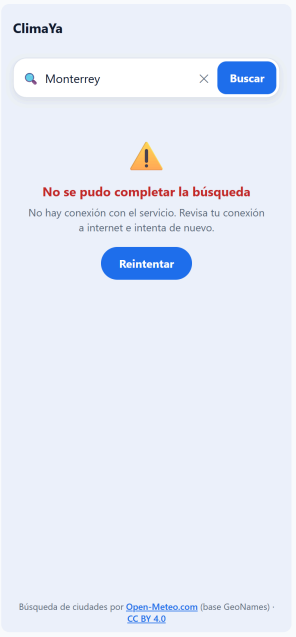
CP01-05-pronostico-7-dias
CP-01 paso 5: pronóstico de 7 días y atribución



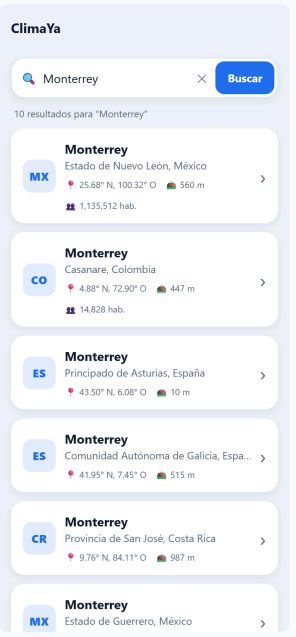
CP01-06-actualizado
CP-01 paso 6: datos actualizados



CP02-03-aviso-sin-conexion
CP-02 paso 3: aviso sin conexión, datos conservados



CP02-04-error-busqueda
CP-02 paso 4: error en búsqueda con Reintentar



CP02-05-recuperado
CP-02 paso 5: recuperación al reconectar

5. Capturas de pantalla para demostrar el funcionamiento

Las capturas finales están en **`evidencias/capturas/`**, numeradas en el orden en que conviene presentarlas. Se tomaron el 31/10/2026 con la app en vista móvil (390 × 844 px) y datos reales de Open-Meteo.

Carpetas de apoyo:

- `evidencias/casos-de-prueba/`: capturas de cada paso de CP-01 y CP-02, con el registro HTTP `registro-ejecucion.json`.
- `evidencias/verificacion/`: capturas de la verificación de punta a punta completa (23 comprobaciones).

5.1 Funcionamiento principal

#	Archivo	Qué muestra	Qué demuestra	Requisito
01	`01-pantalla-inicial.png`	Pantalla de búsqueda con mensaje de bienvenida y ciudades sugeridas	Interfaz móvil intuitiva y estado inicial	Interfaz
02	`02-indicador-de-carga.png`	Spinner "Buscando ciudades… Consultando "Toluca"" y botón Buscar deshabilitado	Indicador de carga durante la solicitud GET	Indicadores de carga
03	`03-resultados-busqueda.png`	"10 resultados para "Toluca"" con tarjetas	Búsqueda en la API y **≥ 3 campos por resultado**: nombre, estado, país, coordenadas, altitud y población	Búsqueda · JSON · ≥ 3 campos
04	`04-clima-actual.png`	Clima de Toluca de Lerdo: 19 °C, Parcialmente nublado, humedad, viento, precipitación y sensación térmica	Segunda llamada GET y presentación de 6 campos del JSON	Integración API · ≥ 3 campos
05	`05-pronostico-7-dias.png`	Pronóstico de 7 días, hora de medición, botón Actualizar y atribución	Procesamiento de arreglos JSON y atribución CC BY 4.0	JSON · Condiciones de uso
06	`06-actualizacion.png`	Pantalla tras pulsar "Actualizar" (hora "Actualizado en la app" nueva)	Actualización de la información	Actualización
07	`07-busqueda-automatica-acentos.png`	Resultados de "Ciudad de México", escrita letra por letra	Búsqueda automática con retardo (1 sola solicitud) y soporte de acentos	Búsqueda

5.2 Estados vacíos y validaciones

#	Archivo	Qué muestra	Qué demuestra	Requisito
08	`08-estado-vacio.png`	"Sin resultados para "Zzqqxxlugarinexistente"" con sugerencia	Estado vacío cuando la API no encuentra coincidencias	Estados vacíos

#	Archivo	Qué muestra	Qué demuestra	Requisito
09	`09-validacion-longitud.png`	Texto "a" → "Escribe al menos 2 caracteres."	Validación de parámetros antes de llamar a la API	Validaciones
10	`10-validacion-caracteres.png`	Texto `<script>` → "Usa solo letras, números, espacios y los signos . , ' -"	Rechazo de entradas maliciosas (0 solicitudes enviadas)	Seguridad · Validaciones

5.3 Manejo de errores

#	Archivo	Qué muestra	Qué demuestra	Requisito
11	`11-error-conexion-actualizar.png`	Aviso amarillo "No hay conexión… Se muestran los últimos datos obtenidos"	Error de conexión sin perder los datos ni cerrar la app	Errores de conexión
12	`12-error-conexion-busqueda.png`	"No se pudo completar la búsqueda" con botón **Reintentar**	Mensaje de error comprensible y opción de recuperación	Mensajes de error
13	`13-recuperacion-reintentar.png`	Resultados de Monterrey tras reconectar y pulsar Reintentar	Recuperación después del error	Errores de conexión
14	`14-limite-solicitudes-429.png`	"Se alcanzó el límite de solicitudes. Espera 30 segundos…"	Manejo de HTTP 429 (simulado, porque no se debe saturar la API real)	Límites de solicitudes
15	`15-respuesta-inesperada.png`	"Se recibió una respuesta inesperada del servicio…"	Manejo de una respuesta que no es JSON (simulada)	Respuestas inesperadas
16	`16-parametros-ruta-invalidos.png`	"No se puede mostrar el clima — La ubicación seleccionada no es válida."	Validación de parámetros manipulados (`lat=999&lon=abc`) sin llamar a la API	Seguridad · Validaciones

5.4 Integración con la API y pruebas

#	Archivo	Qué muestra	Qué demuestra	Requisito
17	`17-api-respuesta-geocodificacion.png`	Solicitud `GET https://geocoding-api…/v1/search?name=Toluca…` , `HTTP 200` , `content-type: application/json` y JSON. Para la captura se recortó a 2 de los 10 resultados (la línea "… (8 resultados más)" es una anotación, no parte de la respuesta)	Uso de HTTPS, método GET y estructura del JSON	Identificación de la API
18	`18-api-respuesta-pronostico.png`	Solicitud `GET https://api.open-meteo.com/v1/forecast…` , `HTTP 200` y JSON completo	Los valores coinciden con la captura 04 (18.7 → 19 °C, 38 %, 6 km/h, `weather_code` 2 → Parcialmente nublado)	Procesamiento de JSON
19	`19-pruebas-unitarias.png`	Salida de `npm test` : 62/62 aprobadas	Validaciones, seguridad y manejo de errores automatizados	Pruebas

#	Archivo	Qué muestra	Qué demuestra	Requisito
20	`20-pruebas-integracion.png`	Salida de `npm run test:integration`: 7/7 aprobadas contra la API real	Pruebas de integración con la API	Pruebas de integración

Las capturas 19 y 20 se generaron a partir del reporte JSON de Jest (`--json`), con el formato de la terminal. Los nombres, resultados y tiempos son los de la ejecución real del 31/10/2026.

5.5 Capturas pendientes en dispositivo físico (recomendadas)

Las capturas anteriores se tomaron en la versión web de la app con vista móvil. Para demostrar que el prototipo corre en un teléfono, toma también estas capturas:

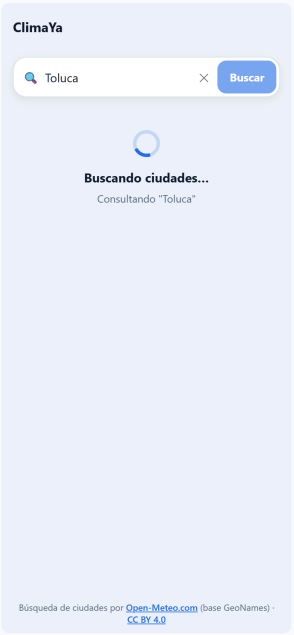
#	Captura a tomar	Cómo obtenerla
F1	Terminal con el código QR de Expo	Ejecuta `npm start` y toma una captura de la terminal
F2	App abierta en Expo Go: pantalla inicial	Escanea el QR con Expo Go; en el teléfono, botón de encendido + bajar volumen
F3	Resultados de búsqueda de "Toluca"	Equivalente a la captura 03
F4	Clima de Toluca de Lerdo	Equivalente a la captura 04
F5	Gesto "deslizar para actualizar" en curso	Desliza hacia abajo en la pantalla de clima y captura mientras gira el indicador
F6	Aviso sin conexión con el modo avión activado	Equivalente a la captura 11; que se vea el ícono de avión en la barra de estado

Guárdalas en `evidencias/capturas/dispositivo/` con el prefijo `F1-`, `F2-`, etc.

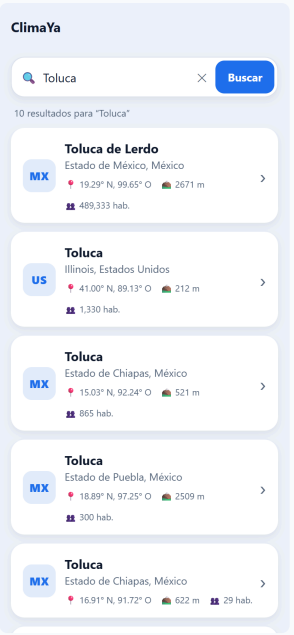
Galería de capturas



01-pantalla-inicial
Pantalla de búsqueda con mensaje de bienvenida y ciudades sugeridas



02-indicador-de-carga
Spinner "Buscando ciudades... Consultando "Toluca"" y botón Buscar deshabilitado



03-resultados-busqueda
"10 resultados para "Toluca"" con tarjetas



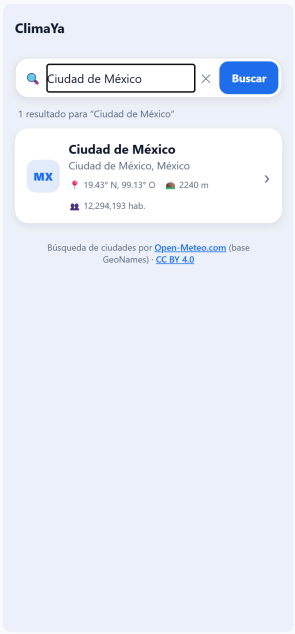
04-clima-actual
Clima de Toluca de Lerdo: 19 °C, Parcialmente nublado, humedad, viento, precipitación y sensación térmica



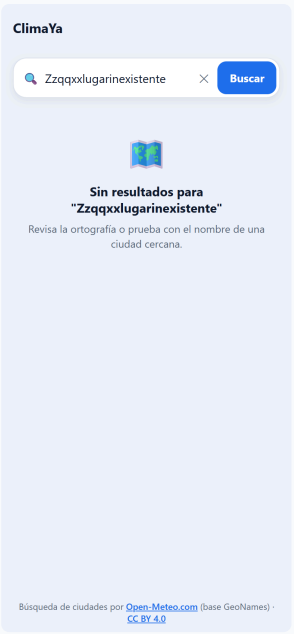
05-pronostico-7-dias
Pronóstico de 7 días, hora de medición, botón Actualizar y atribución



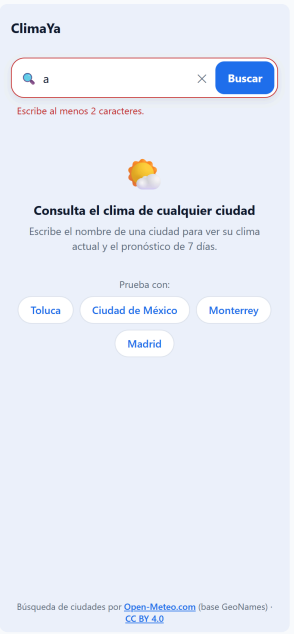
06-actualizacion
Pantalla tras pulsar "Actualizar" (hora "Actualizado en la app" nueva)



07-busqueda-automatica-acentos
Resultados de "Ciudad de México", escrita letra por letra



08-estado-vacio
"Sin resultados para "Zzqqxxlugarinexistente"" con sugerencia



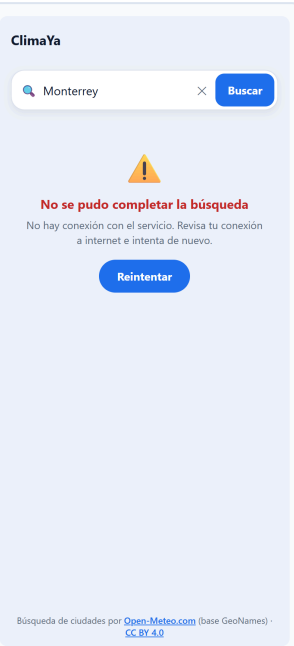
09-validacion-longitud
Texto "a" → "Escribe al menos 2 caracteres."



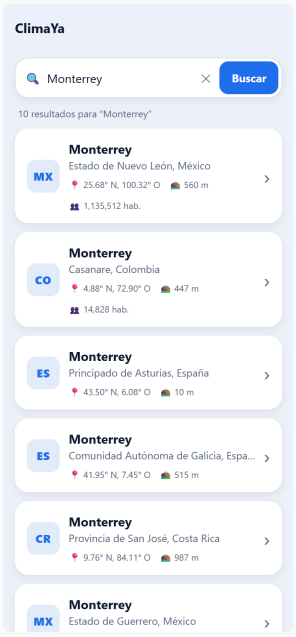
10-validacion-caracteres
Texto <script> → "Usa solo letras, números, espacios y los signos . , ' -"



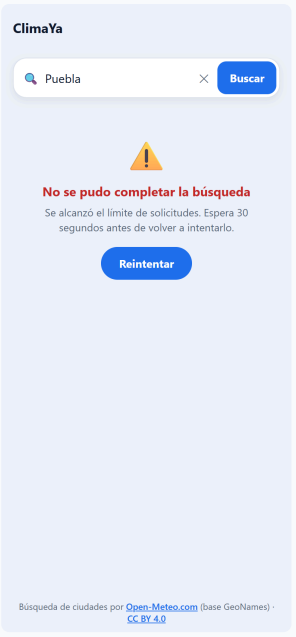
11-error-conexion-actualizar
Aviso amarillo "No hay conexión... Se muestran los últimos datos obtenidos"



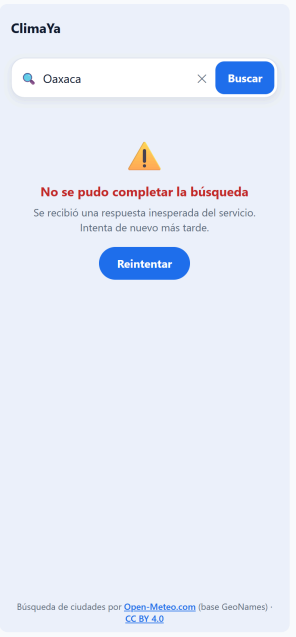
12-error-conexion-busqueda
"No se pudo completar la búsqueda" con botón Reintentar



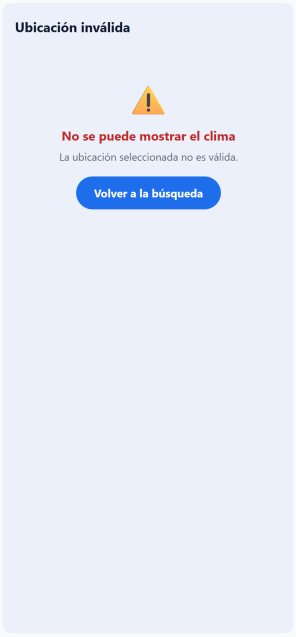
13-recuperacion-reintentar
Resultados de Monterrey tras reconectar y pulsar Reintentar



14-limite-solicitudes-429
"Se alcanzó el límite de solicitudes. Espera 30 segundos..."



15-respuesta-inesperada
"Se recibió una respuesta inesperada del servicio..."



16-parametros-ruta-invalidos
"No se puede mostrar el clima — La ubicación seleccionada no es válida."

Open-Meteo Geocoding API — respuesta real obtenida el 31/10/2026, 10:24:18 a.m.

```
GET https://geocoding-api.open-meteo.com/v1/search?name=Toluca&count=10&language=es&format=json

HTTP 200 OK
content-type: application/json; charset=utf-8
date: Sat, 31 Oct 2026 16:24:19 GMT

{
  "results": [
    {
      "id": 3515302,
      "name": "Toluca de Lerdo",
      "latitude": 19.28786,
      "longitude": -99.65324,
      "elevation": 2671,
      "feature_code": "PPLA",
      "country_code": "MX",
      "admin1_id": 3523272,
      "admin2_id": 8583687,
      "timezone": "America/Mexico_City",
      "population": 489333,
      "country_id": 3996063,
      "country": "México",
      "admin1": "Estado de México",
      "admin2": "Toluca de Lerdo"
    },
    {
      "id": 4913760,
      "name": "Toluca",
      "latitude": 41.00226,
      "longitude": -89.13342,
      "elevation": 212,
      "feature_code": "PPL",
      "country_code": "US",
      "admin1_id": 4896861,
      "admin2_id": 4901299,
      "admin3_id": 4884505,
      "timezone": "America/Chicago",
      "population": 1330,
      "postcodes": ["61369"],
      "country_id": 6252001,
      "country": "Estados Unidos",
      "admin1": "Illinois",
      "admin2": "Condado de Marshall",
      "admin3": "Township of Bennington"
    },
    "… (8 resultados más)"
  ],
  "generationtime_ms": 0.31387806
}
```

17-api-respuesta-geocodificacion
Solicitud GET https://geocoding-api…/v1/search?name=Toluca…, HTTP 200, content-type: application/json y JSON. Para la captura se recortó a 2 de los 10 resultados (la línea "… (8 resultados más)" es una anotación, no parte de la respuesta)

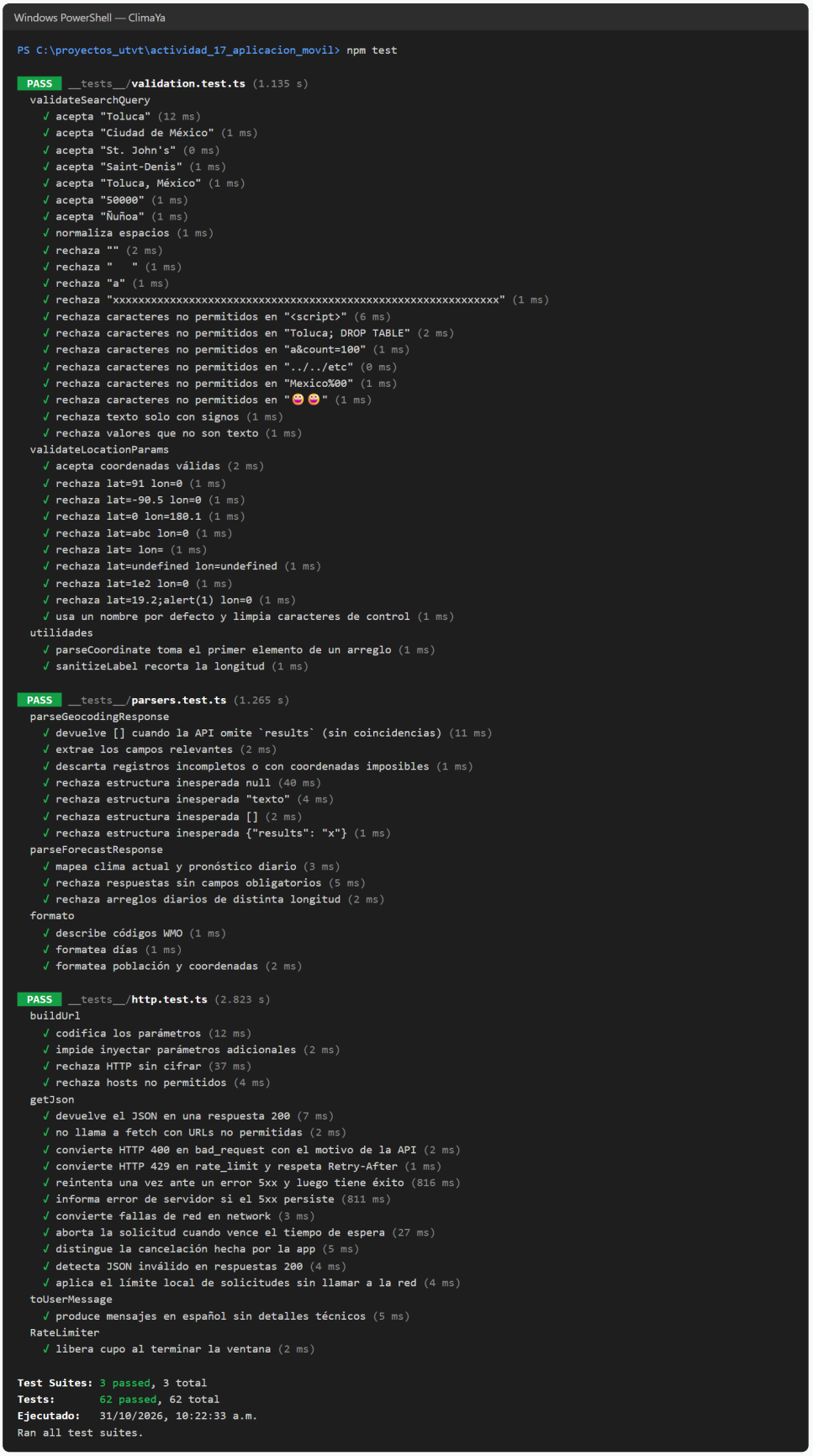
Open-Meteo Forecast API — respuesta real obtenida el 31/10/2026, 10:24:20 a.m.

```
GET https://api.open-meteo.com/v1/forecast?
latitude=19.2879&longitude=-99.6532&current=temperature_2m,relative_humidity_2m,apparent_temperature,is_day,precipitation,
weather_code,wind_speed_10m&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezon
e=auto&forecast_days=7

HTTP 200 OK
content-type: application/json; charset=utf-8
date: Sat, 31 Oct 2026 16:24:21 GMT

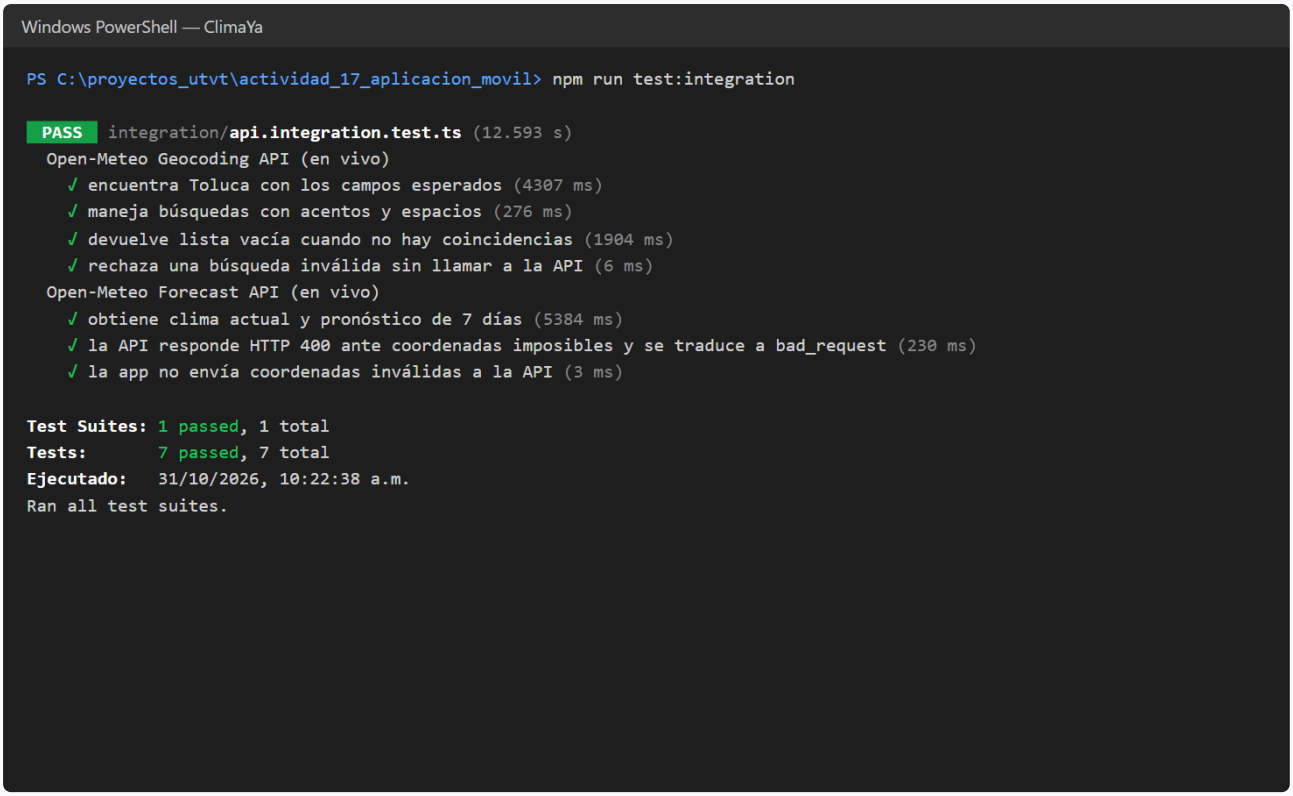
{
  "latitude": 19.226713,
  "longitude": -99.61423,
  "generationtime_ms": 0.5350112915039062,
  "utc_offset_seconds": -21600,
  "timezone": "America/Mexico_City",
  "timezone_abbreviation": "GMT-6",
  "elevation": 2663,
  "current_units": {
    "time": "iso8601",
    "interval": "seconds",
    "temperature_2m": "°C",
    "relative_humidity_2m": "%",
    "apparent_temperature": "°C",
    "is_day": "",
    "precipitation": "mm",
    "weather_code": "wmo code",
    "wind_speed_10m": "km/h"
  },
  "current": {
    "time": "2026-10-31T10:15",
    "interval": 900,
    "temperature_2m": 18.7,
    "relative_humidity_2m": 38,
    "apparent_temperature": 17.5,
    "is_day": 1,
    "precipitation": 0,
    "weather_code": 2,
    "wind_speed_10m": 6
  },
  "daily_units": {
    "time": "iso8601",
    "weather_code": "wmo code",
    "temperature_2m_max": "°C",
    "temperature_2m_min": "°C",
    "precipitation_probability_max": "%"
  },
  "daily": {
    "time": ["2026-10-31", "2026-11-01", "2026-11-02", "2026-11-03", "2026-11-04", "2026-11-05", "2026-11-06"],
    "weather_code": [3, 2, 1, 2, 1, 2, 80],
    "temperature_2m_max": [23.4, 22.8, 23.2, 24.3, 23.7, 23, 20.8],
    "temperature_2m_min": [10, 9.1, 8.9, 10.3, 9.1, 9.4, 10.6],
    "precipitation_probability_max": [0, 2, 0, 0, 12, 33, 59]
  }
}
```

18-api-respuesta-pronostico
Solicitud GET https://api.open-meteo.com/v1/forecast..., HTTP 200 y JSON completo



19-pruebas-unitarias

Salida de npm test: 62/62 aprobadas



20-pruebas-integracion
Salida de npm run test:integration: 7/7 aprobadas contra la API real

6. Documento de entrega

6.1 Datos de identificación

Campo	Valor
Institución	*[completar]*
Asignatura	*[completar]*
Actividad	Actividad 17 — Prototipo de aplicación móvil que consume una API pública
Alumno(a)	*[completar]*
Matrícula / Grupo	*[completar]*
Docente	*[completar]*
Fecha de entrega	*[completar]*
Nombre del prototipo	**ClimaYa** v1.0.0

6.2 Resumen

ClimaYa es un prototipo móvil hecho con **Expo (React Native) y TypeScript** que consume la API pública y gratuita de **Open-Meteo** mediante solicitudes **HTTP GET por HTTPS**. Permite **buscar ciudades** y **consultar y actualizar** su clima actual y su pronóstico de 7 días. Incluye indicadores de carga, estados vacíos, mensajes de error, validación de parámetros y manejo de errores de conexión, respuestas inesperadas y límites de solicitudes. La API no requiere clave, así que la app no contiene credenciales.

Resultados de verificación (31/10/2026): 62/62 pruebas unitarias, 7/7 pruebas de integración contra la API real, 23/23 comprobaciones de punta a punta, y casos de prueba CP-01 (16/16) y CP-02 (6/6) **aprobados**. TypeScript, ESLint y `expo-doctor` sin errores. El bundle de Android compila correctamente.

6.3 Contenido de la entrega

```
actividad_17_aplicacion_movil/
├── README.md                        Instalación, ejecución y uso
├── docs/
│   ├── 01-analisis-y-arquitectura.md Requerimientos, escenarios, stack y arquitectura
│   ├── 02-identificacion-api.md      Identificación y documentación de la API
│   ├── 03-seguridad.md               Medidas de seguridad
│   ├── 04-casos-de-prueba.md         Casos de prueba CP-01 y CP-02
│   ├── 05-capturas-de-pantalla.md    Capturas que demuestran el funcionamiento
│   └── 06-entrega.md                 Este documento
├── evidencias/
│   ├── capturas/                     20 capturas finales numeradas
│   ├── casos-de-prueba/              Capturas por paso + registro HTTP (JSON)
│   └── verificacion/                 Capturas de la verificación completa
├── src/                              Código fuente de la app
├── __tests__/  integration/          Pruebas unitarias y de integración
├── scripts/                          Generación del PDF y del ZIP de entrega
├── app.json  package.json  tsconfig.json  jest*.config.js  eslint.config.js
└── entrega/
    ├── ClimaYa-Documentacion.pdf     Documentación consolidada con capturas
    └── ClimaYa-codigo-fuente.zip     Proyecto sin node_modules
```

6.4 Matriz de cumplimiento

Requisito de la actividad	Cómo se cumple	Evidencia
Analizar requerimientos y seleccionar escenario y API	Tabla de requerimientos y comparación de 5 opciones; se eligió Open-Meteo	`docs/01` §1.1–1.2
Investigar la documentación oficial de la API	Endpoints, parámetros, códigos de error y límites verificados en la documentación oficial y con llamadas reales	`docs/02`
Definir y justificar arquitectura y tecnologías	Stack justificado, diagrama de capas y estructura	`docs/01` §1.3–1.4
Crear el proyecto y configurar dependencias	Proyecto Expo SDK 58 creado desde cero; `expo-doctor` 20/20	`package.json`, `README.md`
Interfaz móvil intuitiva y moderna	Dos pantallas con tarjetas, sugerencias, íconos de clima y navegación con botón atrás	Capturas 01, 03, 04, 05
Integrar la API mediante HTTP GET	`getJson()` con `method: 'GET'` a 2 endpoints	`src/api/http.ts`; capturas 17 y 18; registro HTTP de CP-01
Procesar JSON y mostrar ≥ 3 campos por resultado	7 campos por ciudad y 6 campos de clima actual, más el pronóstico diario	Capturas 03, 04, 05; CP-01 paso 3 y 5
Búsqueda / actualización de información	Búsqueda manual y automática; actualización con botón y gesto	Capturas 03, 06, 07; CP-01 paso 6
Indicadores de carga, estados vacíos y mensajes de error	Componente `StateView` y aviso no bloqueante	Capturas 02, 08, 11, 12
Ejecución en dispositivo o emulador	Bundle Android (Hermes) compilado; app ejecutable con Expo Go	`README.md` §Ejecución; ver limitación 1

Requisito de la actividad	Cómo se cumple	Evidencia
Usar HTTPS	Solo `https://` y lista blanca de hosts	`docs/03` S1–S2; captura 17; prueba unitaria
Evitar exponer claves o credenciales	API sin clave; sin secretos en el código	`docs/03` S3
Validaciones de parámetros	Texto, coordenadas y parámetros de ruta	`docs/03` S4–S6; capturas 09, 10, 16
Manejar errores de conexión, respuestas inesperadas y límites	Errores tipados, reintento, timeout, limitador local, caché y HTTP 429	`docs/03` S7–S11; capturas 11–15; CP-02
Verificar condiciones de uso y atribución	Uso no comercial; atribución CC BY 4.0 en ambas pantallas	`docs/02` §2.2; captura 05
Pruebas de integración con la API	7 pruebas contra la API real	`integration/`; captura 20
Dos casos de prueba completos	CP-01 y CP-02 con todos los elementos solicitados	`docs/04`
Identificar capturas de pantalla	20 capturas clasificadas por requisito y 6 recomendadas en teléfono	`docs/05`

6.5 Instrucciones rápidas para el evaluador

```
npm install                  # 1. Instalar dependencias
npm start                    # 2. Escanear el QR con Expo Go (o: npm run web)
npm test                     # 3. Pruebas unitarias (62)
npm run test:integration     # 4. Pruebas contra la API real (7, requiere internet)
```

Flujo sugerido para la demostración: buscar **Toluca** → tocar **Toluca de Lerdo** → revisar el clima → **Actualizar** → activar el modo avión → **Actualizar** (aparece el aviso) → volver y buscar otra ciudad (aparece el error con **Reintentar**) → desactivar el modo avión → **Reintentar**.

6.6 Limitaciones conocidas y mejoras futuras

1. **Prueba en dispositivo físico pendiente.** Las pruebas de interfaz se hicieron con la versión web en vista móvil, porque el equipo de desarrollo no tiene emulador Android. El bundle nativo de Android compila sin errores, pero falta ejecutarlo en Expo Go y tomar las capturas F1–F6 (`docs/05` §5.5).
2. **Errores 429 y JSON inválido simulados.** Se simularon interceptando la red, porque provocarlos en la API real implicaría abusar del servicio. El manejo con respuestas reales de la API se comprobó para HTTP 200 y HTTP 400.
3. **Caché solo en memoria.** Se pierde al cerrar la app; no hay modo sin conexión persistente.
4. **Uso no comercial.** El plan gratuito de Open-Meteo no permite publicar la app con fines comerciales sin contratar un plan de pago.
5. **Mejoras posibles:** ciudades favoritas guardadas en el dispositivo, clima por ubicación GPS (requiere permiso), pronóstico por hora y modo oscuro.

6.7 Lista de verificación antes de entregar

- [ ] Completar los datos de identificación (§6.1) y regenerar el PDF con `npm run docs:pdf`.
- [ ] Ejecutar la app en un teléfono con Expo Go y tomar las capturas F1–F6.
- [ ] Llenar la tabla "Repetición en dispositivo físico" de `docs/04`.

- [ ] Regenerar el ZIP con `npm run entrega:zip` (incluye el PDF y las capturas nuevas) y revisar que abre e instala con `npm install`.
- [ ] Subir el PDF y el ZIP a la plataforma de la asignatura.

Anexo A. Instalación y ejecución

Requisitos

Herramienta	Versión probada
Node.js	22.x (mínimo 20.19)
npm	11.x
Teléfono con **Expo Go** (Android o iOS)	Compatible con Expo SDK 58

No se necesita Android Studio ni Xcode para ejecutarlo con Expo Go.

Instalación

```
cd actividad_17_aplicacion_movil
npm install
```

Ejecución

Opción A — En un teléfono físico (recomendada)

1. Instala **Expo Go** desde Google Play o la App Store.
2. Conecta la computadora y el teléfono a la **misma red Wi-Fi**.
3. Ejecuta:

   ```
   npm start
   ```

4. Escanea el código QR que aparece en la terminal (en Android, desde Expo Go; en iOS, con la cámara).

Si la red de la escuela bloquea la conexión entre dispositivos, usa un túnel: `npx expo start --tunnel`.

Opción B — En un emulador Android

Con Android Studio y un emulador ya configurados: `npm run android`.

Opción C — En el navegador (para demostración)

```
npm run web
```

Abre http://localhost:8081. Para que se vea como en un teléfono, usa las herramientas de desarrollador del navegador (F12 → vista de dispositivo móvil).

Scripts disponibles

Comando	Descripción
`npm start`	Inicia el servidor de desarrollo de Expo (código QR)
`npm run android` / `npm run web`	Abre la app en un emulador Android o en el navegador
`npm test`	Ejecuta las pruebas unitarias (sin red)
`npm run test:integration`	Ejecuta las pruebas de integración contra la API real (requiere internet)
`npm run typecheck`	Verifica los tipos de TypeScript
`npm run lint`	Analiza la calidad del código con ESLint
`npm run docs:pdf`	Genera `entrega/ClimaYa-Documentacion.pdf` a partir de `docs/` y `evidencias/` (requiere Edge o Chrome)
`npm run entrega:zip`	Genera `entrega/ClimaYa-codigo-fuente.zip` sin `node_modules`

Uso

1. En la pantalla **ClimaYa**, escribe el nombre de una ciudad (p. ej. *Toluca*) o toca una sugerencia.
2. Toca un resultado para ver su clima.
3. En la pantalla de clima, toca ↻ **Actualizar** o desliza hacia abajo para obtener datos nuevos.
4. Usa la flecha ← para volver a la búsqueda.